Projektuppgift

DT071G, Programmering i C#.NET

TaskManager
Projektuppgift i programmering i C#.NET

Francisco De Leon Gonzalez

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi
Författare: Francisco De Leon Gonzalez, frde2500@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026



Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

Sammanfattning

Syftet med projektet var att utveckla en enkel konsolbaserad applikation som hjälper användaren att organisera personliga uppgifter. Applikationen, TaskManager, har implementerats i programmeringsspråket C# med .NET och bygger på objektorienterad programmering. Användaren kan registrera uppgifter med titel, beskrivning och prioritet, visa registrerade uppgifter, redigera information, markera uppgifter som slutförda, ta bort uppgifter och filtrera dem efter status. För att informationen ska finnas kvar mellan programkörningar används lokal lagring i JSON-format genom biblioteket System.Text.Json.

Programmet har delats upp i en klass som representerar en uppgift, en serviceklass för uppgiftshantering och en programfil som hanterar användarens menyval. Under utvecklingen har funktionerna kontrollerats genom manuell körning i konsolen. Samtliga planerade funktionstester genomfördes manuellt och visade att applikationen fungerade enligt de uppställda målen. Även lagringen i JSON-format verifierades genom att uppgifterna fanns kvar efter att programmet avslutats och startats igen.

Resultatet är en fungerande grund för uppgiftshantering utan behov av databas eller grafiskt gränssnitt. Projektet visar hur listor, metoder, villkorssatser, undantagshantering och filhantering kan kombineras i en mindre applikation. Möjliga vidareutvecklingar omfattar bättre validering av prioritet, sökning, sortering och stöd för förfallodatum.

Förord

Denna rapport har skrivits inom ramen för kursen Programmering i C#.NET vid Mittuniversitetet under höstterminen 2026. Projektets syfte har varit att utveckla en konsolbaserad applikation för uppgiftshantering med hjälp av C# och .NET.

Arbetet har gett mig möjlighet att tillämpa kunskaper från kursen, framför allt inom objektorienterad programmering, datastrukturer, filhantering och felhantering. Genom att utveckla en applikation från grunden har jag även fått en större förståelse för hur olika delar av ett program samverkar.

Rapporten redogör för de tekniker och metoder som har använts, hur applikationen har konstruerats samt vilka resultat som har uppnåtts.

Innehållsförteckning

Sammanfattning ii
Förord iii
Terminologi vi
1 Introduktion 1
1.1 Bakgrund och problemmotivering 1
1.2 Övergripande syfte 1
1.3 Avgränsningar 1
1.4 Konkreta och verifierbara mål 1
1.5 Översikt 2
1.6 Författarens bidrag 2
2 Bakgrundsmaterial 3
2.1 C# och .NET 3
2.2 Objektorienterad programmering 3
2.3 Listor och kontrollstrukturer 3
2.4 JSON och filhantering 3
3 Metod 4
3.1 Utvecklingsmiljö och verktyg 4
3.2 Utvecklingsprocess 4
3.3 Datalagring 4
3.4 Testmetod 5
4 Konstruktion 6
4.1 Programstruktur 6
4.2 Användargränssnitt 6
4.3 Programflöde 7
4.4 Modell och serviceklass 7
4.5 Validering och index 8
4.6 Exempel på programkod 8
4.7 Lagring och felhantering 8
5 Resultat 9
5.1 Implementerade funktioner 9
5.2 Testresultat 9
5.3 Resultat i förhållande till målen 9

6 Diskussion 10
6.1 Måluppfyllelse och begränsningar 10
6.2 Möjlig vidareutveckling 10
6.3 Slutsatser 10
Källförteckning 11

Terminologi

Akronymer/Förkortningar

.NET	Plattform för utveckling och körning av applikationer.
C#	Programmeringsspråket som används i projektet.
JSON	JavaScript Object Notation, textformat för strukturerad data.
OOP	Objektorienterad programmering.
CRUD	Create, Read, Update, Delete – grundläggande operationer för datahantering.
Serialisering	Omvandling av objekt till ett format som kan lagras eller överföras.
Deserialisering	Omvandling av lagrad data tillbaka till objekt.

1 Introduktion

1.1 Bakgrund och problemmotivering

Att hålla reda på flera uppgifter kan vara svårt när informationen finns utspridd i anteckningar eller bara sparas i minnet. En enkel uppgiftshanterare kan ge en samlad överblick över vad som behöver göras och vad som redan är klart. Projektet utgår från behovet av ett lättanvänt program som kan köras lokalt utan konto eller internetanslutning.

1.2 Övergripande syfte

Syftet med projektet är att utveckla en enkel och användarvänlig applikation som gör det möjligt att organisera och hantera personliga uppgifter. Applikationen ska ge användaren en tydlig överblick över vilka uppgifter som behöver genomföras och vilka som redan har slutförts.

Projektet syftar även till att tillämpa kunskaper inom programmering i C# och .NET genom att utveckla en fungerande applikation med objektorienterad struktur, användarinmatning och lokal datalagring.

En central del av arbetet är att undersöka hur en mindre applikation kan organiseras med separata klasser och metoder för att skapa en tydlig och underhållbar programstruktur.

1.3 Avgränsningar

Applikationen är textbaserad och avsedd för en användare på en lokal dator. Den har inget grafiskt gränssnitt, inga användarkonton, ingen synkronisering och ingen databas. Prioritet anges som text och kontrolleras ännu inte mot en fast lista över tillåtna värden.

1.4 Konkreta och verifierbara mål

- M1: Användaren ska kunna lägga till en uppgift med titel, beskrivning och prioritet.
- M2: Användaren ska kunna visa, redigera, markera som klar och ta bort uppgifter.
- M3: Användaren ska kunna filtrera uppgifter efter slutförd/ej slutförd.
- M4: Uppgifter ska lagras i JSON och läsas in vid nästa programstart.
- M5: Programmet ska hantera ogiltiga uppgiftsnummer och tomma titlar med meddelanden.

1.5 Översikt

Rapporten är indelad i sex huvudkapitel. Kapitel 1 introducerar projektets bakgrund, syfte, avgränsningar och mål. Kapitel 2 behandlar de teoretiska grunderna för utvecklingen, däribland C#, .NET, objektorienterad programmering och JSON.

Kapitel 3 beskriver den metod som har använts vid utveckling och testning av applikationen. Kapitel 4 redogör för programmets konstruktion, klassstruktur, funktioner och användargränssnitt.

I kapitel 5 presenteras applikationens resultat och genomförda tester. Slutligen innehåller kapitel 6 en diskussion om projektets måluppfyllelse, begränsningar och möjligheter till vidareutveckling.

1.6 Författarens bidrag

Projektet har genomförts som ett individuellt arbete. Jag har ansvarat för projektets inriktning, utvecklingsmiljö, implementering, körning och kontroll av applikationens funktioner samt dokumentation av resultatet.

Under utvecklingen har jag använt AI-baserat stöd för att få förklaringar av programmeringsbegrepp, förslag på kodlösningar och hjälp med felsökning. AI-stöd har även använts vid utformningen av rapportens text och struktur. Applikationens funktionalitet har kontrollerats genom körning i Visual Studio.

2 Bakgrundsmaterial

2.1 C# och .NET

C# är ett programmeringsspråk som används för att utveckla applikationer inom .NET. .NET erbjuder bland annat körmiljö och standardbibliotek för inmatning, filhantering och datastrukturer [1]. I projektet används Console.ReadLine för inmatning och Console.WriteLine för utskrift.

2.2 Objektorienterad programmering

Objektorienterad programmering innebär att program kan organiseras med klasser och objekt. TaskItem beskriver data för en uppgift, medan TaskManagerService ansvarar för funktioner som ändrar och visar uppgifterna. Denna uppdelning gör koden lättare att läsa och vidareutveckla [1].

2.3 Listor och kontrollstrukturer

Uppgifterna lagras under körning i List<TaskItem>. En lista gör det möjligt att lägga till och ta bort objekt samt iterera över dem. Programmet använder en while-loop för huvudmenyn, switch för menyval och if-satser för validering.

2.4 JSON och filhantering

JSON är ett textbaserat format för strukturerad information. Biblioteket System.Text.Json kan serialisera C#-objekt till JSON och deserialisera JSON till objekt [2]. TaskManager sparar uppgifterna i tasks.json och läser in filen vid start.

3 Metod

3.1 Utvecklingsmiljö och verktyg

Applikationen TaskManager har utvecklats med programmeringsspråket C# och ramverket .NET. Visual Studio har använts som utvecklingsmiljö för att skriva, organisera, kompilera och köra programkoden.

För versionshantering har Git använts tillsammans med GitHub. Detta har gjort det möjligt att spara förändringar i projektet och publicera programkoden i ett externt kodarkiv.

Applikationen har utvecklats som ett konsolprogram eftersom projektets huvudsakliga fokus har varit att implementera funktionalitet och tillämpa objektorienterad programmering snarare än att utveckla ett grafiskt användargränssnitt.

3.2 Utvecklingsprocess

Utvecklingen har genomförts stegvis genom att applikationens funktioner implementerats och kontrollerats efter hand.

Arbetet inleddes med att definiera vilka funktioner programmet skulle innehålla. Därefter skapades en modellklass för att representera uppgifter och en separat serviceklass för att hantera dessa.

När den grundläggande programstrukturen var på plats implementerades funktioner för att lägga till, visa, markera som slutförda och ta bort uppgifter. Därefter utökades programmet med filtrering, redigering och möjlighet att spara uppgifterna i JSON-format.

Huvudmenyn utvecklades för att användaren skulle kunna komma åt funktionerna genom att ange olika menyval.

3.3 Datalagring

För att lagra användarens uppgifter valdes JSON som filformat. JSON gör det möjligt att spara strukturerad information i en textfil och läsa in den igen vid nästa programstart. Biblioteket System.Text.Json används för att omvandla programmets uppgiftslista till JSON och tillbaka till C#-objekt.

Vid programstart kontrolleras om lagringsfilen finns. Om filen finns läses uppgifterna in i programmets lista. När användaren genomför en förändring sparas den uppdaterade listan till filen. Denna metod valdes eftersom applikationen är avsedd för lokal användning och inte behöver någon extern databas.

3.4 Testmetod

Applikationens funktioner kontrollerades genom manuell testning i konsolen. Testningen innebär att olika menyval används och att programmets beteende jämförs med de förväntade resultaten.

Testfallen omfattar bland annat att skapa, redigera, slutföra och ta bort uppgifter samt att filtrera dem efter status. Även felaktig inmatning, exempelvis ogiltiga uppgiftsnummer och tomma titlar, ingår i testplanen.

För att kontrollera datalagringen avslutas programmet efter att en uppgift har skapats. Programmet startas sedan om för att undersöka om uppgiften fortfarande finns kvar.

Resultaten dokumenterades i en test tabell där det framgår vilka funktioner som har kontrollerats och om de fungerade enligt förväntningarna. Testerna genomfördes manuellt och omfattar inte automatiserade enhetstester.

4 Konstruktion

4.1 Programstruktur

Fil	Ansvar
Program.cs	Visar meny, läser inmatning och anropar serviceklassens metoder.
TaskItem.cs	Modellklass med Title, Description, Priority och IsCompleted.
TaskManagerService.cs	Hanterar lista, skapande, redigering, borttagning, filtrering och JSON.

4.2 Användargränssnitt

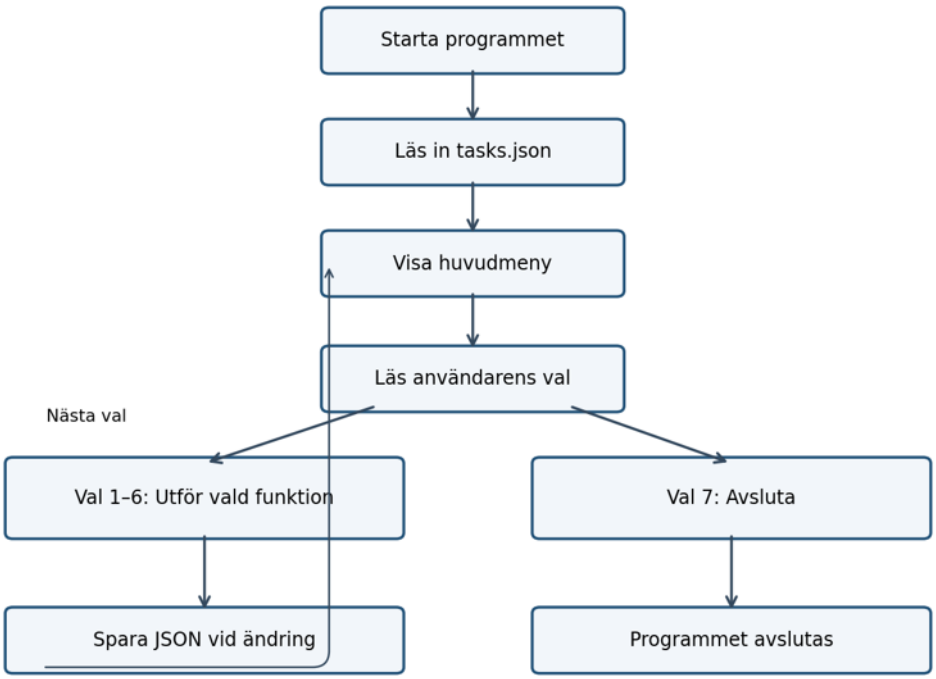
Applikationen använder en numrerad textmeny. Figur 1 visar en förenklad skiss av huvudmenyn, inte en skärmbild från körningen.



Figur 1. Egen skiss över TaskManagers konsolmeny.

4.3 Programflöde

Vid start försöker programmet läsa in tidigare uppgifter. Därefter visas menyn upprepade gånger. Användaren väljer en funktion och återgår sedan till menyn. Ändringar sparas i JSON. Figur 2 visar huvudflödet.



Figur 2. Förenklat flödesschema för TaskManager.

4.4 Modell och serviceklass

Klassen TaskItem innehåller fyra egenskaper: Title, Description, Priority och IsCompleted. Klassen TaskManagerService innehåller en privat lista av TaskItem-objekt. AddTask skapar objekt och lägger dem i listan. EditTask ändrar egenskaperna på ett befintligt objekt. CompleteTask ändrar status till true, medan RemoveTask tar bort objektet från listan.

När användaren väljer en funktion i huvudmenyn anropar Program.cs motsvarande metod i TaskManagerService. Serviceklassen hanterar därefter de TaskItem-objekt som finns i listan och genomför den begärda åtgärden. På detta sätt hålls användarens inmatning åtskild från logiken för uppgiftshanteringen.

4.5 Validering och index

Användaren ser uppgiftsnummer som börjar på 1, medan listans index börjar på 0. Därför skickar Program.cs vidare användarens nummer minus ett. Metoderna CompleteTask, RemoveTask och EditTask kontrollerar att index ligger inom listans gränser. Tom titel avvisas vid skapande och redigering. Ogiltiga menyval ger ett felmeddelande.

4.6 Exempel på programkod

Följande kod visar hur en uppgift markeras som slutförd:

```
public bool CompleteTask(int index)

{

   if (index < 0 || index >= tasks.Count)

      return false;

   tasks[index].IsCompleted = true;

   return true;

}
```

Metoden returnerar false om uppgiftsnumret är ogiltigt och true om status har uppdaterats. Efter en lyckad ändring anropar huvudprogrammet SaveTasks().

4.7 Lagring och felhantering

SaveTasks serialiserar listan med JsonSerializer.Serialize och skriver den till tasks.json med File.WriteAllText. LoadTasks kontrollerar först om filen finns och använder sedan JsonSerializer.Deserialize. Båda metoderna har try/catch för att fånga fel vid filhantering. Om inläsningen misslyckas visas ett felmeddelande.

5 Resultat

5.1 Implementerade funktioner

Applikationen innehåller funktioner för att skapa, visa, redigera, slutföra, ta bort och filtrera uppgifter. JSON-lagring används för att bevara uppgifterna mellan körningar. Dessa funktioner är implementerade i den aktuella programversionen.

5.2 Testresultat

Tabell 1 sammanfattar resultaten från de manuella funktionstester som genomfördes av applikationen TaskManager. Testerna omfattade programmets grundläggande funktioner samt hantering av felaktig användarinmatning. Samtliga testfall genomfördes med förväntat resultat.

Testfall	Förväntat resultat	Status
Skapa uppgift	Ny uppgift visas i listan	Verifierat
Redigera uppgift	Ändrad titel/beskrivning/prioritet visas	Verifierat
Markera klar	Status ändras till Klar	Verifierat
Ta bort uppgift	Uppgiften försvinner	Verifierat
Filtrera status	Endast vald status visas	Verifierat
Avsluta och starta om	Sparad uppgift finns kvar	Verifierat
Ogiltigt uppgiftsnummer	Felmeddelande visas	Verifierat
Tom titel	Uppgift skapas inte	Verifierat

Tabell 1. Resultat från manuella funktionstester av TaskManager.

5.3 Resultat i förhållande till målen

Resultatet av den manuella testningen visar att applikationens implementerade funktioner fungerar enligt de uppställda målen. Användaren kan skapa, visa, redigera, slutföra, ta bort och filtrera uppgifter. Även lagringen i JSON-format har verifierats genom att uppgifter finns kvar efter att programmet har avslutats och startats igen.

Testresultaten visar därmed att målen M1–M5 har uppnåtts. Applikationen uppfyller projektets grundläggande krav och kan användas för lokal uppgiftshantering.

6 Diskussion

6.1 Måluppfyllelse och begränsningar

TaskManager visar att en mindre konsolapplikation kan byggas med tydlig uppdelning mellan användargränssnitt, datamodell och verksamhetslogik. JSON gör att programmet kan bevara data utan en databas. Samtidigt är användargränssnittet enkelt och bygger på att användaren skriver in siffror och text. Prioritet lagras som fri text, vilket innebär att inkonsekventa värden kan förekomma.

En fördel med den valda programstrukturen är att användargränssnittet och uppgiftshanteringen är uppdelade i separata delar. Det innebär att förändringar i programmets meny inte nödvändigtvis kräver förändringar i hur uppgifterna lagras och hanteras. Samtidigt innebär användningen av en lokal JSON-fil vissa begränsningar, exempelvis att programmet inte är anpassat för flera användare eller synkronisering mellan olika enheter. För projektets omfattning bedömdes dock denna lösning vara tillräcklig eftersom fokus låg på att utveckla en fungerande applikation och tillämpa objektorienterad programmering.

6.2 Möjlig vidareutveckling

En fortsatt utveckling kan omfatta validering av prioritet, sortering och sökning, förfallodatum och en grafisk användarmiljö. Automatiserade enhetstester skulle också göra det lättare att kontrollera att funktionerna fortsätter fungera efter kodändringar.

6.3 Slutsatser

Projektets huvudsakliga resultat är en fungerande konsolbaserad uppgiftshanterare som använder objektorienterad programmering, listor, användarinmatning och JSON-lagring. Genom manuell testning har det verifierats att applikationens funktioner fungerar enligt de uppställda målen. Resultatet visar att det är möjligt att utveckla en enkel och användbar applikation för uppgiftshantering utan att använda en databas eller ett grafiskt användargränssnitt.

Projektet har även bidragit till en ökad förståelse för hur olika delar av en C#-applikation samverkar, särskilt när det gäller klassstruktur, metoder, filhantering och felhantering.

Källförteckning

[1] Microsoft. A tour of C# – Overview [Internet]. Microsoft Learn. Tillgänglig: https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/overview Hämtad 2026-10-26

[2] Microsoft. System.Text.Json overview [Internet]. Microsoft Learn. Tillgänglig: https://learn.microsoft.com/dotnet/standard/serialization/system-text-json/overview Hämtad 2026-10-26